

归并排序

归并排序是利用递归与分治的技术将数据序列划分为越来越小的半子表，再对半子表排序，最后再用递归方法将排好序的半子表合并成越来越大的有序序列。

关键：第一步:划分半子表； 第二步：合并半子表。

原理：对于给定的一组记录（假设有 n 个记录），首先将每两个相邻的长度为 1 的子序列进行归并，得到 $n/2$ (向上取整)个长度为 2 或者 1 的有序子序列，再将其两两归并，反复执行此过程，直到得到一个有序序列。

示例代码：

```
inline void msort(int l,int r)
{   if (l==r) return;
    int i,j,u=l-1,m=(l+r)>>1;
    msort(l,m),msort(m+1,r);
    i=l,j=m+1;
    while ((i<=m)&&(j<=r))
        if (a[i]<a[j]) b[++u]=a[i++];
        else b[++u]=a[j++];
    while (i<=m) b[++u]=a[i++];
    while (j<=r) b[++u]=a[j++];
    for (i=l;i<=r;i++) a[i]=b[i];
}
```

快速排序

快速排序是 C.R.A.Hoare 于 1962 年提出的一种划分交换排序。它采用了一种分治的策略，通常称其为分治法(Divide-and-ConquerMethod)。

快速排序是对归并排序算法的优化，继承了归并排序的优点，同样应用了分治思想。所谓的分治思想就是对于一个问题“分而治之”，用分治思想来解决问题需要两个步骤：

如何“分”？（如何缩小问题的规模）

如何“治”？（如何解决子问题）

快排的前身是归并，而正是因为归并存在不可忽视的缺点，才产生了快排。归并的最大问题是需要额外的存储空间。针对这一点，快速排序提出了新的思路：把更多的时间用在“分”上，而把较少的时间用在“治”上。从而解决了额外存储空间的问题，并提升了算法效率。

快排之所以被称为“快”排，是因为它在平均时间上说最快的，主要原因是硬件方面的，每趟快排需要指定一个“支点”（也就是作为分界点的值），一趟中涉及的所有比较都是与这个“支点”来进行比较的，那么我们可以把这个“支点”放在寄存器里，如此这般，效率自然大大提高。除此之外，快排的高效率与分治思想也是分不开的。

说白了就是给基准数据找其正确索引位置的过程。

首先用一个临时变量去存储基准数据,比如说取第一个数;然后分别从数组的两端扫描数组，设两个指示标志:low 指向起始位置，high 指向末尾。

首先从后半部分开始，如果扫描到的值大于基准数据就让 high 减 1,如果发现有元素比该基准数据的值小，就将 high 位置的值赋值给 low 位置：

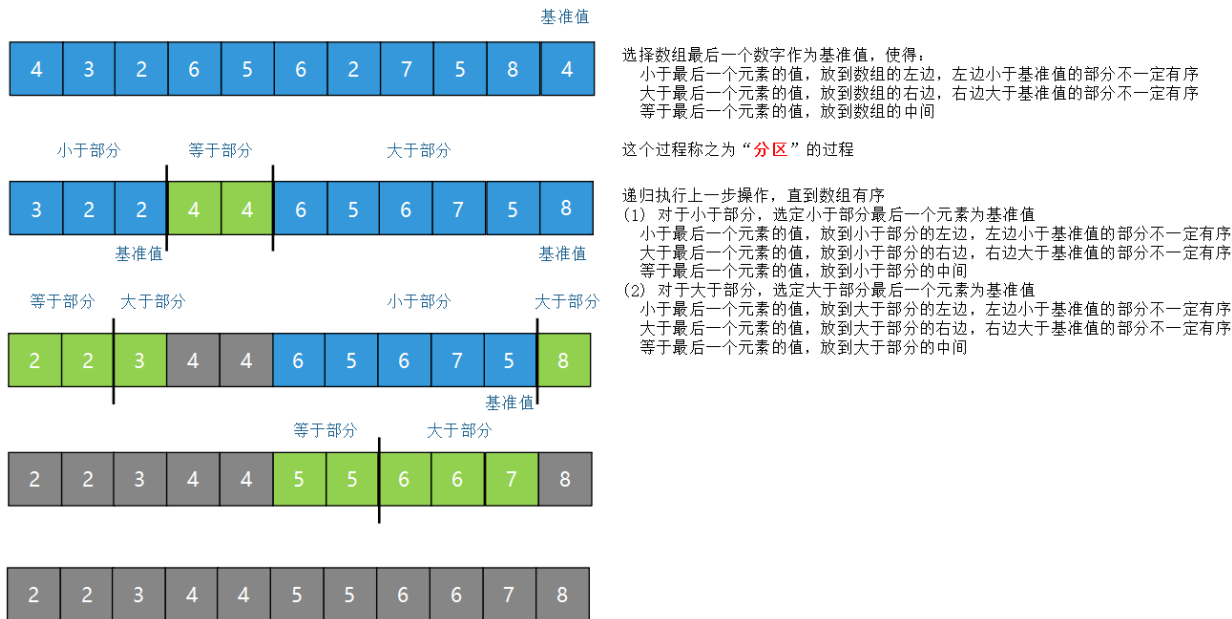
然后开始从前往后扫描,如果扫描到的值小于基准数据就让 low 加 1,如果发现有元素大于基准数据的值，就再将 low 位置的值赋值给 high 位置的值：

然后再开始从后向前扫描,原理同上。

然后再开始从后向前扫描,原理同上。

以此循环遍历,直到 low=high 结束循环,此时 low 或 high 的下标就是基准数据在该数组中的正确索引位置.这样一遍走下来,可以很清楚的知道,其实快速排序的本质就是把基准数大的都放在基准数的右边,把比基准数小的放在基准数的左边,这样就找到了该数据在数组中的正确位置。

之后采用递归的方式分别对前半部分和后半部分排序，当前半部分和后半部分均有序时该数组就自然有序了。



以上过程可参见 [qsort2.gif](#)。

从上面的过程中可以看到：

①先从队尾开始向前扫描且当 $low < high$ 时,如果 $a[high] > tmp$,则 $high--$,但如果 $a[high] < tmp$,则将 $high$ 的值赋值给 low ,即 $arr[low] = a[high]$,同时要转换数组扫描的方式,即需要从队首开始向队尾进行扫描了

②同理,当从队首开始向队尾进行扫描时,如果 $a[low] < tmp$,则 $low++$,但如果 $a[low] > tmp$ 了,则就需要将 low 位置的值赋值给 $high$ 位置,即 $arr[low] = arr[high]$,同时将数组扫描方式换为由队尾向队首进行扫描。

③不断重复①和②,知道 $low \geq high$ 时(其实是 $low = high$), low 或 $high$ 的位置就是该基准数据在数组中的正确索引位置。

快速排序的分区方法有很多，这里再举一种，该方法的基本思想是：先从数列中取出一个数作为基准数；分区过程，将比这个数大的数全放到它的右边，小于或等于它的数全放到它的左边；再对左右区间重复第二步，直到各区间只有一个数。

示例代码：

```
inline void kp(int l,int r)
{   int t,i=l,j=r,m=a[(l+r)>>1];
    while (i<=j)
    {   while ((i<=j)&&(m>a[i])) i++;
        while ((i<=j)&&(m<a[j])) j--;
        if (i<=j)
        {   t=a[i],a[i]=a[j],a[j]=t;
            i++,j--;
        }
    }
    if (l<j) kp(l,j);
    if (i<r) kp(i,r);
}
```

快速排序和归并排序是互补的：归并排序是将数组分成两个子数组分别排序，并将有序的子数组归并以将整个数组排序，而快速排序将数组排序的方式是当两个子数组都有序时整个数组也就有序了。